

Automated Produce Scanner with Realtime Pricing and Packaging

Core Objective: To design a self-service kiosk that identifies vegetables via computer vision (using EDGE IMPULSE STUDIO), weighs (using LOAD SENSOR WITH HX711 module) them, calculates price, and provides secure packaging(using an IMPULSE SEALER AS THE HEATING ELEMENT and 12 LINEAR ACTUATOR AS THE MECHANICAL ARM that will push and pull the other arm of the Impulse sealer).

Hardware Specification & Connectivity:

Component	Specification	Function / Role
Microcontroller	Raspberry Pi 4 Model B (4 GB RAM) Trixie Version <i>a.k.a rpi 4</i>	Handles OS (Debian 13), Model Inference, and GPIO control.
Vision Sensor	RPI Camera Module V2 (IMX219)	Captures real-time frames at the weighing platform. Supporting Details: Using the EDGE IMPULSE STUDIO, we have trained it to create a ".eim" file that will serve as the file that handles the offline inferencing. It is stored locally in the rpi 4 and had given permission to be executable for offline use and referencing in the offline inference code. We have captured 1000 plus onion images to trained the model (.eim file) and labeled it as a ONION. The other variable that is labeled NOT ONION are the captured images of common object or elements such as hand, chargers, pencils, etc.
Weight Sensor	Load Cell + HX711 Amplifier	Detects weight in grams.
User Interface	16x2 LCD with I2C Module	Real-time display for Weight and Price
Inputs	Physical Push Button	Tare functionality and Sealing/Barcode trigger. Supporting Details: There are two tactile switch or push buttons in the

		system or device. We will call them as <i>TARE button</i> and the <i>SEAL/BARCODE button</i>. 1. The tare button: It serve as a tare function of the system. As part of the system is like a smart weighing scale, if this button is pressed, it will tare the whole weighing system. 2. The seal/barcode button: Will serve as a confirmation to the customer. If this button is pressed, the sealing mechanism will function or proceed. Using the actuator as the mechanical arm that will push and pull the other arm of the impulse sealer. At the same time if the seal/barcode button will pressed, the barcode generation and printing will also execute.
Sealer	Impulse Heat Sealer Machine for Plastic Packaging Vacuum Sealer Machine 100mm link: https://shopee.ph/Impulse-Heat-Sealer-Machine-for-Plastic-Packaging-Vacuum-Sealer-Machine-100mm-150mm-200mm-250mm300mm-i.128307891.5548257462?extraParams=%7B%22display_model_id%22%3A70719831997%2C%22model_selection_logic%22%3A3%7D&sp_atk=e30b80bf-51ad-4f5f-aa82-00bbb73bf0d2&xptdk=e30b80bf-51ad-4f5f-aa82-00bbb73bf0d2	Serve as the sealer using its heat element to seal the plastic that contains the vegetable such as onion or gralic or marble potato.

Barcode scanner	LogicOwl OJ-910 Barcode Scanner with Stand 1D Reader for POS System, Store, Supermarket, Warehouse Link: https://shopee.ph/Logicowl-OJ-910-Wired-USB-Type-Portable-1D-Barcode-Scanner-for-POS-P2P-JG-Superstore-i.15328948.7609697057?extraParams=%7B%22display_model_id%22%3A32048420455%2C%22model_selection_logic%22%3A3%7D&sp_atk=bb6cf2d4-36f4-44fa-b21d-2a39e80604bd&xptdk=bb6cf2d4-36f4-44fa-b21d-2a39e80604bd	Serve as the component that will scan the printed barcode
Thermal Printer	Compact Thermal Printer Module for Weighing Apparatus and Self-Service Terminal Link: https://shopee.ph/Compact-Thermal-Printer-Module-for-Weighing-Apparatus-and-Self-Service-Terminal-i.52659352.23583679379?extraParams=%7B%22display_model_id%22%3A128774221171%2C%22model_selection_logic%22%3A3%7D&sp_atk=fbc5e52c-c61d-4f5c-83fd-9cfe55e25b70&xptdk=fbc	Will serve as the printer of the generated barcode.

	5e52c-c61d-4f5c-83fd-9cfe55e25b70&fbclid=IwY2xjawQqbX1leHRuA2FlbQlXMQBzcnRjBmFwcF9pZAEwAAEeqMwz72g_SkSOMnY58T6z8ewDNsLIJKzfZmsDdLk6vITICwmhvzaHUhW1Me8_aem_3BF5K2Akh4U4k7rAJ_nXzw	
Mechanical Arm	Electric Linear Actuator DC 12V 1500N Aluminum alloy Linear Actuator Motor IP54 Link: https://shopee.ph/product/756087807/22169662786?d_id=c2651&uls_trackid=5570ndlp00kt&utm_content=3c3hzP7UWgepAXPji3EU79boUKr7	Serves as the mechanical Arm that pushes and pulls the other arm of the impulse sealer. When the seal/barcode button is pressed, this mechanical arm (linear actuator) will function.
Power supply		

Overall Functionality:

The device uses an image classification technology that uses the edge impulse studio technology for producing and training the “.eim file” or the “.eim executable” later to be used in the inference code that is python language.

Below is the working offline inference code that was utilized or used in the system:

INFERENCE CODE :

```
#!/usr/bin/env python3
import json
import time
import statistics
from collections import deque

import cv2
import numpy as np
from picamera2 import Picamera2
from edge_impulse_linux.image import ImageImpulseRunner

from HX711 import SimpleHX711, Mass, Options, ReadType, Value
from RPLCD.i2c import CharLCD

# =====
# USER SETTINGS
# =====
MODEL_PATH = "/home/jekaca/virtual_environment/jekaca_sama-project-1-linux-aarch64-v2.eim"

MODEL_WIDTH = 96
MODEL_HEIGHT = 96

# Classification behavior
CONFIDENCE_THRESHOLD = 0.80
SMOOTHING_FRAMES = 5
INFER_INTERVAL = 0.25      # faster inference cycle like code 1
CAMERA_WARMUP_SEC = 1.0

# Camera capture size
CAPTURE_WIDTH = 640
CAPTURE_HEIGHT = 480
```

Pricing

PRICE_PER_KG_PHP = 100.0

VALID_PRODUCT_LABELS = None

EMPTY_LIKE_LABELS = {"empty", "background", "none", "no object"}

=====

HX711 / GPIO CONFIG

=====

DT_PIN = 6

SCK_PIN = 5

CAL_FILE = "hx711_calibration.json"

=====

LCD CONFIG

=====

LCD_I2C_ADDRESS = 0x27

LCD_I2C_PORT = 1

LCD_COLS = 16

LCD_ROWS = 2

=====

STABLE WEIGHT SETTINGS

=====

RAW_READS_PER_SAMPLE = 5

FILTER_WINDOW = 3

WEIGHT_READ_INTERVAL = 0.20 # separate weight timing like code 1 style

ZERO_DEADBAND_G = 1.0

NEGATIVE_CLAMP = True

AUTOZERO_BAND_G = 2.0

AUTOZERO_STABLE_RANGE_G = 0.5

AUTOZERO_CONSECUTIVE = 4

AUTOZERO_STEP_RAW = 1

WARMUP_READS = 5

```

# =====
# UI SETTINGS
# =====

WINDOW_NAME = "Onion Detection Preview"
PREVIEW_WIDTH = 800
PREVIEW_HEIGHT = 600

# Optional detection debounce to avoid flicker
ONION_ON_CONSEC = 2
ONION_OFF_CONSEC = 2

# =====
# HELPER FUNCTIONS
# =====

def load_cal():
    with open(CAL_FILE, "r", encoding="utf-8") as f:
        return json.load(f)

def init_lcd():
    lcd = CharLCD(
        i2c_expander='PCF8574',
        address=LCD_I2C_ADDRESS,
        port=LCD_I2C_PORT,
        cols=LCD_COLS,
        rows=LCD_ROWS,
        dotsize=8
    )
    lcd.clear()
    return lcd

def lcd_show(lcd, line1="", line2=""):
    lcd.home()
    lcd.write_string(f"{line1[:LCD_COLS]:<{LCD_COLS}}")
    lcd.crlf()
    lcd.write_string(f"{line2[:LCD_COLS]:<{LCD_COLS}}")

def clamp_and_deadband(grams: float) -> float:

```

```
if abs(grams) <= ZERO_DEADBAND_G:
    grams = 0.0
if NEGATIVE_CLAMP and grams < 0:
    grams = 0.0
return grams
```

```
def read_raw_median(hx: SimpleHX711) -> int:
    return int(hx.read(Options(RAW_READS_PER_SAMPLE, ReadType.Median)))
```

```
def center_crop_to_square(img_bgr: np.ndarray) -> np.ndarray:
    h, w = img_bgr.shape[:2]
    side = min(h, w)
    y0 = (h - side) // 2
    x0 = (w - side) // 2
    return img_bgr[y0:y0 + side, x0:x0 + side]
```

```
def model_view_from_frame(frame_bgr: np.ndarray) -> np.ndarray:
    sq = center_crop_to_square(frame_bgr)
    model_img = cv2.resize(
        sq,
        (MODEL_WIDTH, MODEL_HEIGHT),
        interpolation=cv2.INTER_AREA
    )
    return model_img
```

```
def format_price_php(price: float) -> str:
    return f"P{price:.2f}"
```

```
def is_valid_product(label: str, conf: float) -> bool:
    if conf < CONFIDENCE_THRESHOLD:
        return False

    low = label.strip().lower()

    if low in EMPTY_LIKE_LABELS:
        return False

    if VALID_PRODUCT_LABELS is not None:
```

```

        return low in {x.lower() for x in VALID_PRODUCT_LABELS}

    return True

def draw_text_block(img, text, org, scale=0.7, thickness=2):
    x, y = org
    cv2.putText(
        img, text, (x, y),
        cv2.FONT_HERSHEY_SIMPLEX,
        scale, (0, 0, 0), thickness + 3, cv2.LINE_AA
    )
    cv2.putText(
        img, text, (x, y),
        cv2.FONT_HERSHEY_SIMPLEX,
        scale, (255, 255, 255), thickness, cv2.LINE_AA
    )

def build_ui_frame(frame_bgr, status_text, detected_name, weight_g, price_php, conf_pct):
    display = cv2.resize(frame_bgr, (PREVIEW_WIDTH, PREVIEW_HEIGHT),
        interpolation=cv2.INTER_LINEAR)

    # simpler panel for better speed
    cv2.rectangle(display, (10, 10), (440, 170), (40, 40, 40), -1)

    draw_text_block(display, f"Status: {status_text}", (25, 40), scale=0.75, thickness=2)
    draw_text_block(display, f"Detected: {detected_name}", (25, 75), scale=0.68, thickness=2)
    draw_text_block(display, f"Weight: {weight_g:.2f} g", (25, 110), scale=0.68, thickness=2)
    draw_text_block(display, f"Price: PHP {price_php:.2f}", (25, 145), scale=0.68, thickness=2)
    draw_text_block(display, f"Conf: {conf_pct:.1f}%", (25, 175), scale=0.62, thickness=2)
    draw_text_block(display, "Q = Quit", (660, 35), scale=0.6, thickness=2)

    return display

# =====
# MAIN
# =====

def main():

```

```
print("Starting vegetable detection + weighing + LCD + smoother realtime preview...")
print("Press Q on the preview window to stop.\n")
```

```
# -----
```

```
# Load HX711 calibration
```

```
# -----
```

```
cal = load_cal()
```

```
ref_unit_int = int(cal["ref_unit_int"])
```

```
base_offset = int(cal["offset"])
```

```
hx = SimpleHX711(DT_PIN, SCK_PIN)
```

```
hx.setUnit(Mass.Unit.G)
```

```
hx.setReferenceUnit(Value(ref_unit_int))
```

```
hx.setOffset(Value(base_offset))
```

```
for _ in range(WARMUP_READS):
```

```
    read_raw_median(hx)
```

```
    time.sleep(0.03)
```

```
dynamic_offset = int(base_offset)
```

```
gram_window = deque(maxlen=FILTER_WINDOW)
```

```
near_zero_stable_counter = 0
```

```
# -----
```

```
# LCD
```

```
# -----
```

```
lcd = init_lcd()
```

```
lcd_show(lcd, "System Ready", "Loading model")
```

```
# -----
```

```
# Edge Impulse runner
```

```
# -----
```

```
runner = ImageImpulseRunner(MODEL_PATH)
```

```
model_info = runner.init()
```

```
labels = model_info["model_parameters"]["labels"]
```

```
print("Model labels:", labels)
```

```
# -----
```

```

# Camera
# -----
picam2 = Picamera2()
config = picam2.create_preview_configuration(
    main={"size": (CAPTURE_WIDTH, CAPTURE_HEIGHT), "format": "RGB888"},
    buffer_count=4
)
picam2.configure(config)
picam2.start()
time.sleep(CAMERA_WARMUP_SEC)

lcd_show(lcd, "Camera Ready", "Starting...")

# -----
# OpenCV window
# -----
cv2.namedWindow(WINDOW_NAME, cv2.WINDOW_NORMAL)
cv2.resizeWindow(WINDOW_NAME, PREVIEW_WIDTH, PREVIEW_HEIGHT)

# -----
# Inference smoothing/state
# -----
conf_history = deque(maxlen=SMOOTHING_FRAMES)
last_infer_time = 0.0
last_weight_time = 0.0
last_lcd_text = ("", "")

last_onion_conf = 0.0
last_avg_conf = 0.0

onion_state = False
onion_on_count = 0
onion_off_count = 0

current_label = "No object"
current_weight_g = 0.0
current_price = 0.0

```

try:

while True:

now = time.time()

=====

ALWAYS GET FRAME FIRST

This keeps preview responsive like code 1

=====

frame_rgb = picam2.capture_array()

frame_bgr = cv2.cvtColor(frame_rgb, cv2.COLOR_RGB2BGR)

=====

INFERENCE ONLY EVERY INTERVAL

=====

if (now - last_infer_time) >= INFER_INTERVAL:

last_infer_time = now

model_img = model_view_from_frame(frame_bgr)

features, _ = runner.get_features_from_image(model_img)

result = runner.classify(features)

classification = result.get("result", {}).get("classification", {})

onion_conf = float(classification.get("onion", 0.0))

conf_history.append(onion_conf)

avg_conf = sum(conf_history) / len(conf_history)

last_onion_conf = onion_conf

last_avg_conf = avg_conf

detected = avg_conf >= CONFIDENCE_THRESHOLD

Debounce ON/OFF state like code 1

if detected:

onion_on_count += 1

onion_off_count = 0

else:

onion_off_count += 1

```

onion_on_count = 0

if (not onion_state) and onion_on_count >= ONION_ON_CONSEC:
    onion_state = True
elif onion_state and onion_off_count >= ONION_OFF_CONSEC:
    onion_state = False
    current_weight_g = 0.0
    current_price = 0.0

if onion_state:
    current_label = "onion"
else:
    current_label = "No object"

print("-" * 60)
print(f"Raw onion conf : {last_onion_conf*100:.1f}%")
print(f"Avg onion conf : {last_avg_conf*100:.1f}%")
print(f"State      : {current_label}")

# =====
# WEIGHT ONLY WHEN ONION IS DETECTED, RATE-LIMITED
# =====
if onion_state and (now - last_weight_time) >= WEIGHT_READ_INTERVAL:
    last_weight_time = now

    raw = read_raw_median(hx)
    grams = (raw - dynamic_offset) / ref_unit_int

    gram_window.append(grams)
    filtered_grams = statistics.median(gram_window) if gram_window else grams

if len(gram_window) == FILTER_WINDOW:
    window_min = min(gram_window)
    window_max = max(gram_window)
    window_span = window_max - window_min

    if abs(filtered_grams) <= AUTOZERO_BAND_G and window_span <=
AUTOZERO_STABLE_RANGE_G:

```

```

        near_zero_stable_counter += 1
    else:
        near_zero_stable_counter = 0

    if near_zero_stable_counter >= AUTOZERO_CONSECUTIVE:
        if raw > dynamic_offset:
            dynamic_offset += AUTOZERO_STEP_RAW
        elif raw < dynamic_offset:
            dynamic_offset -= AUTOZERO_STEP_RAW

    final_grams = clamp_and_deadband(filtered_grams)
    current_weight_g = float(final_grams)
    current_price = (current_weight_g / 1000.0) * PRICE_PER_KG_PHP

elif not onion_state:
    current_weight_g = 0.0
    current_price = 0.0

# =====
# LCD UPDATE
# =====
if onion_state:
    line1 = "ONION DETECTED"
    line2 = f"{int(round(current_weight_g))}g {format_price_php(current_price)}"
else:
    line1 = "Not onion"
    line2 = "Wt:0g P:0.00"

if (line1, line2) != last_lcd_text:
    lcd_show(lcd, line1, line2)
    last_lcd_text = (line1, line2)

# =====
# UI DISPLAY
# =====
if onion_state:
    status_text = "ONION DETECTED"
    detected_name = "onion"

```

```

        shown_weight = current_weight_g
        shown_price = current_price
    else:
        status_text = "no vegetable..."
        detected_name = "None"
        shown_weight = 0.0
        shown_price = 0.0

    ui_frame = build_ui_frame(
        frame_bgr,
        status_text=status_text,
        detected_name=detected_name,
        weight_g=shown_weight,
        price_php=shown_price,
        conf_pct=last_avg_conf * 100.0
    )

    cv2.imshow(WINDOW_NAME, ui_frame)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        print("\nStopped by user (Q pressed).")
        break

except KeyboardInterrupt:
    print("\nStopped by user.")

finally:
    try:
        runner.stop()
    except Exception:
        pass

    try:
        picam2.stop()
    except Exception:
        pass

```

```
try:
    cv2.destroyAllWindows()
except Exception:
    pass

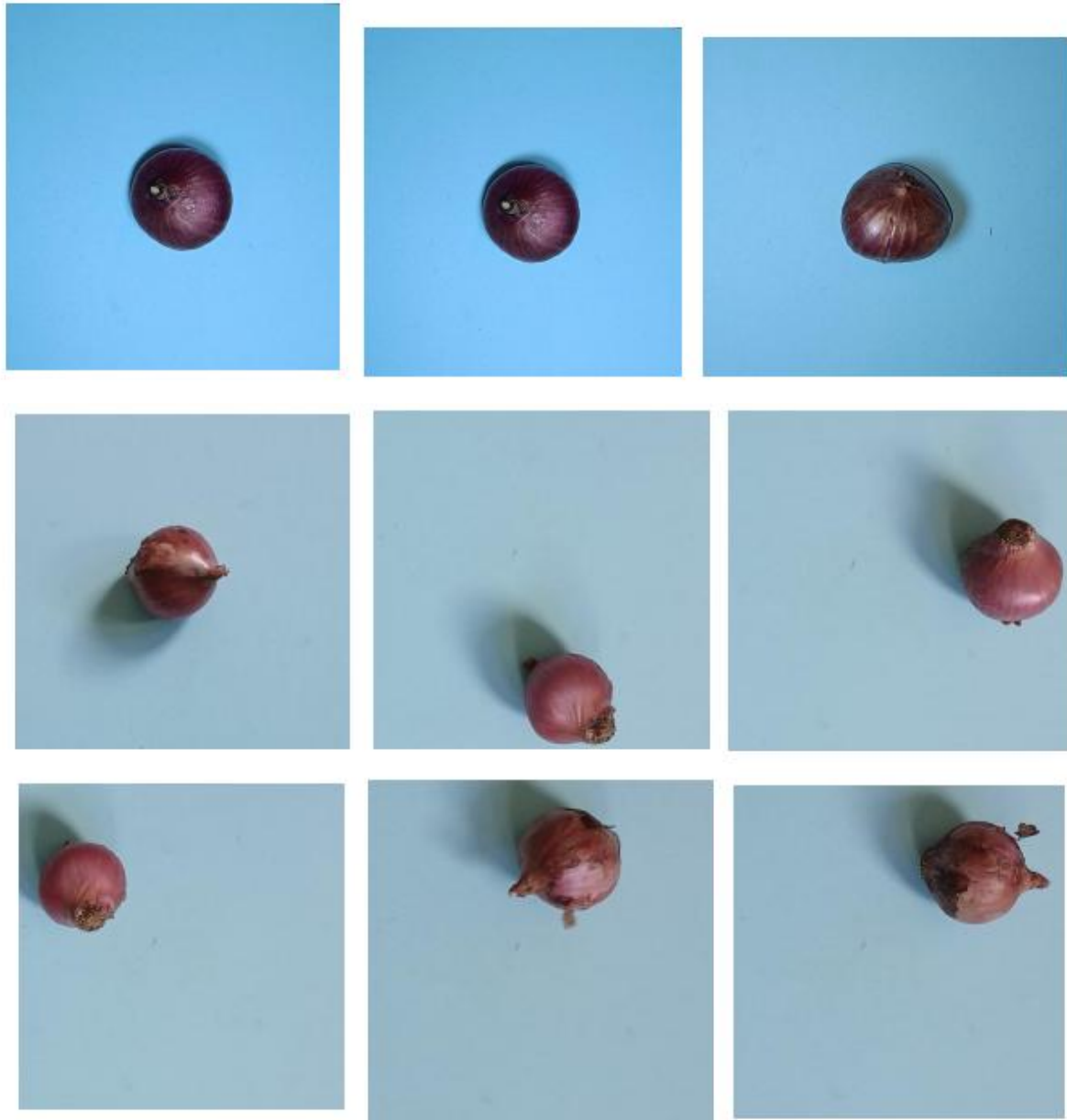
try:
    lcd_show(lcd, "System stopped", "")
    time.sleep(0.5)
    lcd.close(clear=True)
except Exception:
    pass
```

```
if __name__ == "__main__":
    main()
```

The device is targeting to successfully and accurately to identify three unique vegetables. Those vegetables are “ Onion”, “ Garlic”, “ Marble Potato”.

The group member succesfully captured datasets of onions. The quantity of the datasets used to trained the EIM model are comprise of 1200 images of onions.

Below are the sample images of onions in the datasets:



These images are labeled as “ONIONS” in the edge impulse studio.

The group have also pictured common objects such as hands, chargers, etc. And labeled it as a “NOT ONIONS” in the Edge Impulse Studio to make the eim model more accurate and efficient.

Below are the sample datasets of the NOT ONIONS images



NOTE: We have use a constant background that is of SKY BLUE in color so that it will reduce the amount of complications or errors in image classification thus increasing the system accuracy.

The group is still in progress of creating the other data sets such as the garlic, marble potato, and the null object or the common object such as hands, the back ground and other common objects that will be labeled as ***not vegetables*** in the edge impulse studio and in the inference python code.

The group is using the Thonny Python as its integrated development environent or the IDE and running the code in the raspberry pi 4 model B 4 gb Ram.

The system is divided into sections based on its functionality. The sections are the following:

- 1.) Image Classification (subject to change as it will be upgraded to image detection)
2.) Weighing function
3.) Sealing Function
4.) Barcode Generation and Printing Function

Further Details about the sections:

1.) Image Classification / Detection:

Components:

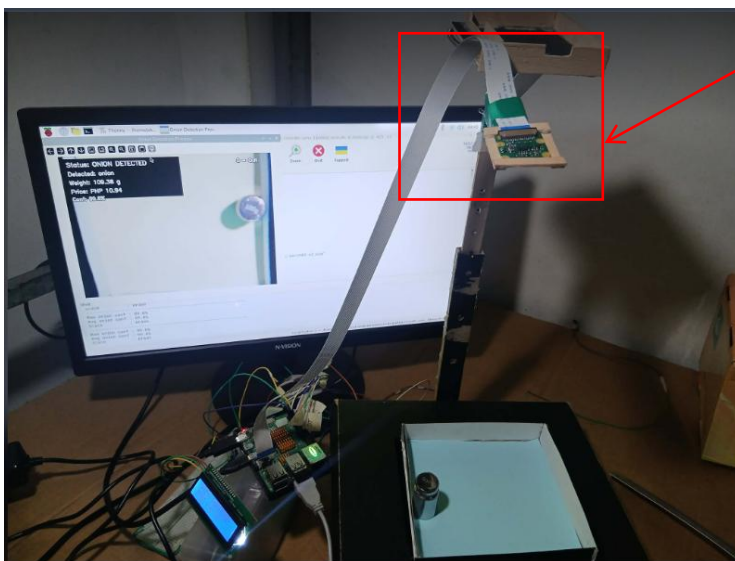
> RPi Camera Module V2 (IMX219)



In the RPi terminal, the camera can be tested using the following command:

`rpikam-hello -t 5000`

Imx219
camera



The function of the camera is to see whatever is on the weigh platform (the sky blue background). Whatever it sees will be then processed by the offline inference code.

The user interface of the system is the lcd module with i2c. In realtime scenario, if the camera detects an onion, the lcd will also output an onion. Otherwise, it will back to default that it will out as no vegetable.





5. Overall, the lcd just serve as the visual indicator for the user about what is happening in realtime.

Pinouts or the connection of the lcd in the rpi 4:

5v to pin 4

Gnd to pin 6

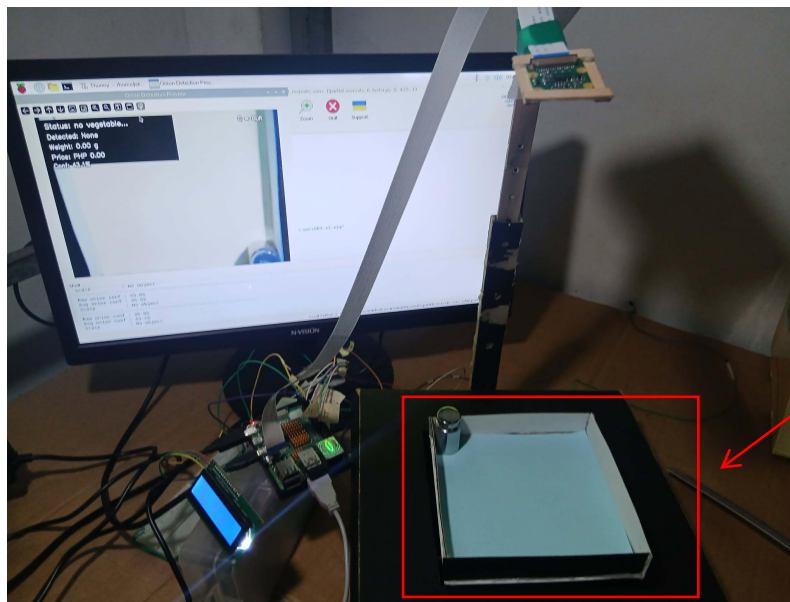
Sda to pin 3 (GPIO 2)

Scl to pin 5 (GPIO 3)

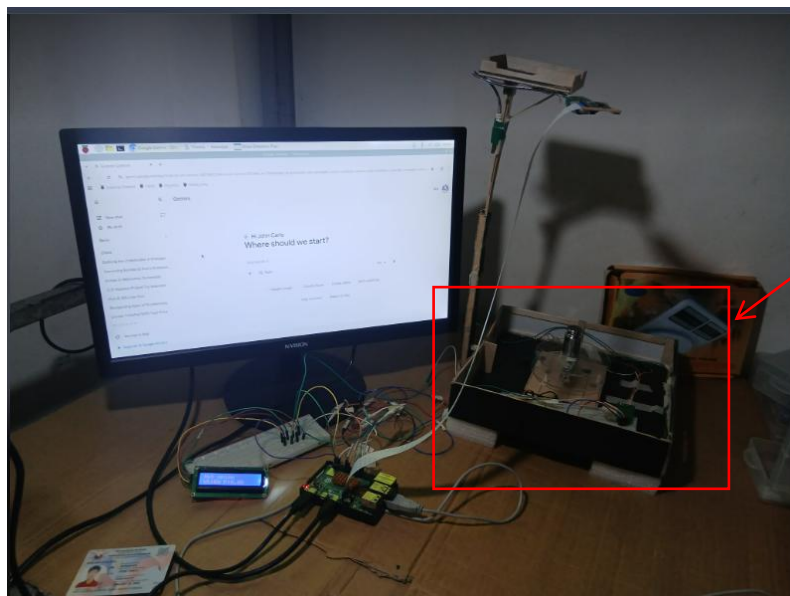
2.) Weighing function

Components:

Load sensor with hx711 amplifier



The weigh platform
(the sky blue
background)



The load sensor
together with hx711
amplifier wired in the
breadboard.

The load sensor with hx711 amplifier does the reading of weight that is put on the weighing platform.

Pinouts or the connection of the load sensor in the rpi 4:

5v to 5v

Gnd to gnd

DT_PIN = GPIO 6

SCK_PIN = GPIO 5

3.) Sealing Function (to do)

components:

>linear actuator

>impulse sealer

The linear actuator will serve as the mechanical arm that pushes the arm of the impulse sealer to seal the plastic after the seal/barcode button is press.

4.) Barcode Generation and Printing Function:

Components

HARDWARE:

> Thermal printer

> Barcode Scanner

SOFTWARE:

> sqlite database (fully offline)

After the seal/barcode button is pressed, the thermal printer will print the barcode that is generated, and then it will be printed on the thermal printer. After that, the user, using the barcode scanner, can scan the barcode that is printed on the paper. After the scanning, it will then search in the sqlite database about the information that was scanned, and it will be then fetched and then be displayed on a certain user interface in the cashier.

The sketch of the device:

